Use Case Justification

The one-line case

The goal is an autonomous agent for **Quote-to-Cash (Q2C)**: one orchestrator per business unit that fronts many workflows (email triage and PO to quote today; escalation, follow-ups, order and invoice handling as we scale), all on one common data platform. Q2C is the first domain, not the boundary: the platform is designed so other functions can adopt the same pattern toward an organization-wide agent. For it to act autonomously it needs two things no individual workflow owns: **shared memory** of what has been seen, decided, and done across workflows and data sources, and **business-curated guardrails and information** that are controlled by existing logic but maintained by the business without a code release. No existing store in our ecosystem is shaped for both, and none spans every system a request touches (Request Central, Jira, Web Quote, Partner First, mail).

Where we are today

The first workflow live under the orchestrator is email triage (Auto Triage BRD v1.2, APPP space; Dev Studio setup pages). Its shape is the template for what follows:

```
Sources (mail via IDP, APIs, CIS apps, NexChat, ...)
        → Event → Workflow (Dev Studio)
        → Orchestrator → sub-agents (PO→Quote, escalation, ticket
create/update, ...)
        → shared MCP → systems of record (RC, Jira, Web Quote, CIS, ...)
```

- **IDP** stores raw payload + per-document metadata, scoped to one *IDP Code* (one pipeline). It is an ingestion edge, not shared memory. **This is the MongoDB service.**
- **Systems of record** (Request Central, Jira, Web Quote, CIS) each hold one slice of a request's story. None is the whole picture, and none holds the agent's context or the rules it operates under. Not every BU or request path uses Request Central.
- **Orchestrator + sub-agents** are stateless between runs: each decision is made from the input in front of them plus whatever the MCP layer can fetch live.

The gaps this creates

Evidence below comes from the Auto Triage BRD, the first workflow under the orchestrator. The same needs recur for every workflow added, which is the point: they belong to the agent, not to any one workflow.

Gap	Evidence in current docs	What a shared DB provides
Shared context / dedup across workflows	Triage scope includes "duplicate detection and linking". Open: "How should duplicate/follow-up matching work when identifiers are missing?" The same request is currently worked in multiple inboxes.	A ledger of everything the agent has seen and done (normalized identifiers, thread/customer keys, linked ticket, quote, or order) that any workflow can query before acting.
Reasoning cost control	D4: two-tier models, deterministic resolution first. Risk: "AI classification cost rises with volume."	Cached prior decisions and lookup tables let more cases resolve with no LLM call, and give the reasoning model prior context instead of re-deriving it.
Workflow-to-workflow context	Future sub-agents named: quote generation, escalation. PO → Quote already follows triage.	Each workflow sees what the others decided (category, confidence, what was already quoted). No point-to-point integration per pair.
Business-owned guardrails	Taxonomy is the "single source of truth"; open: "Who owns the taxonomy after launch?" and "what confidence thresholds…" OKR 3: configure new actions/workflows *without code* in ≥25% of cases.	Taxonomy, routing rules, thresholds, allowed actions, and approval limits as governed, versioned data the business edits, rather than prompts or constants in workflow code.
Learning from human oversight	§5.7: correction outcomes from human review are "captured for continuous improvement", with no specified store.	A corrections table tied to the original decision, so guardrails can be refined from evidence.
Measurement	OKRs are baseline-relative; "measurement design is being roadmapped" (data sources, baseline capture, sampling).	A decision log (action, confidence, path taken, outcome, correction) is the measurement source for every workflow.
Extensibility	OKR 3: onboard a new inbox in <1 day of engineering effort.	New sources, workflows, and actions are configuration rows plus an MCP tool, not new code and a new schema each time.

Tying every source together

Not every request passes through Request Central. Requests arrive or live in Jira, Web Quote, Partner First, NexChat, and mail. A common database does not help unless each of those has a defined way to reach it. The proposed tie (draft):

- **One canonical request record** per customer request or transaction, independent of any source system (the `activity_ledger`).
- **Source references** linking that record to each system's own identifier (RC ticket, Jira key, Web Quote id, Partner First order, mail thread). Duplicates and follow-ups are recognized by matching normalized identifiers *across* sources, not within one.
- **Ingestion by integration, not through a hub system.** Mail arrives through IDP events into Dev Studio workflows. CIS and external applications are reached through APIs or MCP tools. NexChat, the approved CIS agent platform, is a conversational entry point.

Source	Reaches the platform via	Status
Outlook mail	IDP → Event → Dev Studio workflow	Exists (Auto Triage)
Request Central	API / MCP	Mechanism TBD; not used by every BU
Jira	API / MCP	Mechanism TBD
Web Quote	API / MCP	Mechanism TBD
Partner First	Not yet identified	Open
NexChat	Registered APIs / MCP	Mechanism TBD
SharePoint	Not yet identified	Open
Teams	Out of scope for v1: governance for Microsoft collaboration data is unresolved. Requests are visible only where they land in a CIS app or NexChat.	Decision needed

The technical mechanism per source is still to be determined. The need and the shape of the tie are not.

First use case: triage duplicate detection

Spelled out because it is the first thing the database has to do. Proposed flow:

- **Problem:** the same request reaches multiple inboxes and is worked by multiple teams (BRD §2.2).

- **Constraint today:** IDP stores documents per IDP Code and makes only a spam decision. The BRD (D2) puts deduplication in the hosted pipeline, which has no shared place to look.
- **Flow:** IDP receives mail and emits an event. The Dev Studio workflow normalizes identifiers (customer, PO number, quote number, thread) and queries the ledger through MCP or an aggregate service. A match links the email to the existing record and its source references (for example, the RC ticket). No match creates a new record. The orchestrator then continues.
- **Beyond mail:** the same ledger lets a Jira ticket or Web Quote for the same request be recognized as the same request.

Two kinds of data, one schema

The core design point: the DB holds data with different owners and write rules, and they must live together so agents can join across them.

	Logic-controlled	Business-curated
Written by	Agents / workflows (service accounts)	Business users via an admin UI or governed change path
Examples	Processed-request ledger, decision log, confidence scores, links to tickets / quotes / orders, corrections as captured	Taxonomy, routing rules, confidence thresholds, action permissions and approval limits, customer/alias mappings, exception lists
Change control	Schema-enforced; append-mostly; no manual edits	Versioned, approved, auditable; effective-dated
Failure mode if wrong	Bad context for later agents	Misrouted work at scale

Why not use what exists

Existing store	Why it doesn't cover this
RC	System of record for tickets, and not every BU or request path uses it. Not designed as an agent-queryable context/config store, and we shouldn't couple agent internals to a ticketing schema owned by another team (RC PO: Lori Thompson).
Any single source system (RC, Jira, Web Quote)	Each holds one slice of a request. Making one the hub excludes every request that never touches it.

Existing store	Why it doesn't cover this
IDP storage	Per-IDP-Code document payloads and sidecar metadata. Right for ingestion; scoped to one pipeline, and has no concept of cross-workflow state or business-edited config.
Per-workflow stores	Recreates the silo problem. N workflows → N schemas → pairwise integrations; the orchestrator can't reason across them.
Agent Bridge schemas (NDP-5241)	Production precedent for agent data in this ecosystem, but scoped to Agent Bridge (admin catalog, tenants, templates, `DBRG` reseller runtime). Different product, tenancy model, and owner. Useful as a pattern, not a home for Q2C agent data.
Prompts / workflow constants	Where business logic effectively lives today. Needs a release per change, not auditable by the business, not queryable by other agents.

Platform Context (what the internal ecosystem gives us)

Findings from Confluence (space **ID**, plus APPP). Verified against the pages listed under Sources.

Dev Studio

- Internal low-code platform. Projects contain components (IDP, Event, Workflow, decision rule, aggregate service, URL route, MongoDB, ...). Components must be **approved** before **Edit in Studio**, then pushed to Git, bound to a OneTool Deployment Plan (NDP), and promoted DEV → UAT → PROD.
- Auto Triage is already built this way: IDP + Event + Workflow components (pages 793518042 / 793518043).
- New artifact types are added through a documented skill/plugin path (771984648), so Dev Studio has been extended for data components before.

Database options already supported

1. **Relational schema via Dev Studio metadata + DDL deploy objects.**
   - A schema is registered under Metadata → Database Schema and **must be assigned to a Database Group** (name must match the real schema) (605672539).
   - Tables ship as DDL deploy objects (`DO-DDL-TABLE`, CREATE), deployed in category order DDL → ADHOC → K8S → DevStudio, executed by DBA/Ops, with a rollback of "drop only if empty and DBA approves" (NDP-5241, 782639356).
   - ID generation uses the shared **odometer** service.

2. **Dev Studio MongoDB component.** Register schema → Apply to environment → Deploy through the standard flow; CRUD (insert/update/query/delete) goes through `config-deployment-proxy`. Recently enhanced (NDP-7665) and bug-fixed for compound indexes (NDP-8129). Internal-platform risk rated Low.
3. **Decision-rule-managed tables.** The event-driven architecture decision (240338470) manages publisher/subscriber tables through Dev Studio decision rules. That is existing precedent for **business-editable configuration tables backed by the platform's own governance**, directly relevant to the curated half of our schema.

Governance and discoverability

- Databases are owned by a **Database Group** and consumed by named services. The `api-management` MCP server (705296321) can trace ownership and dependencies: services by DB schema/account/URL, service metadata (IT owner, biz domain), and deployment details per environment. A new schema registered correctly becomes discoverable to agents and to ops through this.
- Hyve and Disty (two business environments, separate URLs per DEV/UAT/PROD) mean the schema's **business scope** (Shared vs. Disty) must be chosen deliberately. NDP-5241 splits Agent Bridge into a *Shared* admin schema and a *Disty* (`DBRG`) runtime schema.

Closest precedent: Agent Bridge (NDP-5241 / NDP-8047)

Agent Bridge's initial production release shipped **15 DDL tables** (9 shared admin-catalog tables for tenants/MCP servers/skills/templates/deploy audit, 5 Disty runtime tables including `agent_bridge_approval_request` for human-in-the-loop), plus 3 K8S services and 8 Dev Studio artifacts. This is the path our schema would follow, and it shows the approach clears release governance as a Low-risk, additive change.

Proposed Shape (for discussion)

Draft

A starting position, not a decision. Entity names are placeholders to codesign with the business.

Standalone schema (own Database Group, own owner), reachable from the orchestrator and sub-agents through the **shared MCP layer** (consistent with BRD D3: build connections once, reuse across agents). Agents don't get raw connections; they get scoped MCP tools.

Logical domains:

Domain	Contents	Writer
`activity_ledger`	Canonical record per request or transaction, independent of source system: customer/identifier keys, current state	Agents
`source_refs`	Links each ledger record to its id in every source system (RC ticket, Jira key, Web Quote id, Partner First order, mail thread)	Agents
`decision_log`	Action chosen, category, confidence, path (deterministic / small model / reasoning model / human review), model + prompt version, timestamps	Agents
`correction_log`	Human oversight outcomes linked to decision_log rows	Reviewers → workflow
`guardrails`	Allowed actions per workflow, approval limits, escalation rules, confidence thresholds, effective dates	Business
`taxonomy_routing`	Categories, subcategories, routing rules, queue mapping	Business
`reference_data`	Customer, product, and per-source configuration (e.g. per-inbox settings)	Business

Storage choice. Relational (DDL DO + Database Group) is the better fit for the ledger and curated config: joins across domains, constraints, effective dating, DBA-reviewed change. MongoDB component is a candidate only if we later need to keep large semi-structured payloads (parsed source payloads, model traces) alongside; IDP already holds raw document payloads, so start without it.

Business curation path. Needs a decision: Dev Studio decision-rule/admin UI (precedent exists) vs. a lightweight admin page like Agent Bridge's `agent-admin`. Either way, business edits go through approval and versioning rather than direct SQL.

Requirements the DB must meet

- Standardized, versioned schema reviewed jointly by dev and business.
- Source-agnostic: no request path (RC, Jira, Web Quote, Partner First, mail) is required to go through any one system to be recognized.
- Clear separation of agent-written vs. business-written tables, with different permissions.

- Auditability: who/what changed config and when; every agent decision reproducible from logged inputs and versions.
- Environment parity (DEV/UAT/PROD) and correct Hyve/Disty scoping.
- Read path fast enough for each workflow's SLA (e.g. triage: p95 ingestion-to-action under 5 min, 35 emails/min recommended peak, 70/min backlog recovery). Lookups must not become the bottleneck.
- Retention and privacy aligned with existing policies (e.g. BRD §12 for email content); prefer storing identifiers and decisions over full source content.
- Registered with `api-management` so ownership and dependencies are discoverable.

Risks

Risk	Mitigation
Curated config becomes a second, ungoverned source of truth alongside prompts	Move taxonomy/thresholds out of prompts entirely; single read path via MCP
Business edits mis-route at scale	Approval + effective-dating + rollback to prior version; shadow/UAT validation
Stale or wrong ledger data poisons later agents	Append-only ledger, provenance on every row, TTL/refresh rules
Scope creep into a general data platform	Start with Q2C domains; add others only with a named consuming workflow
Sources with no integration path (Teams, Partner First) leave blind spots in the shared memory	List in-scope sources explicitly; route others through CIS apps or NexChat; report source coverage
PII/customer data accumulation	Store identifiers and decisions, not bodies; follow retention policy
Ownership ambiguity (taxonomy owner is an open BRD question)	Name data owner per domain before first DDL

Open Questions

Not answered in the Confluence pages I found:

1. **Provisioning path:** I found how schemas are registered (Database Group) and how tables ship (DDL DOs), but no page describing how a *new Database Group / physical schema* is requested and who approves it. Confirm with Dev Studio / DBA team. (Likely the NDP-5241 route; verify.)

2. Relational engine and where the physical schema lives for Shared vs. Disty scope.
3. Whether Dev Studio supports a business-facing edit UI for a *custom* table without building an admin app.
4. Who owns each curated domain (taxonomy, thresholds, guardrails) after launch. The taxonomy owner is already an open BRD question and blocks the curated half.
5. Whether Phase 2 workflows (PO→Quote) will live in the same Dev Studio project as Auto Triage or separate projects sharing the schema.
6. Access model: can sub-agents and the orchestrator read via an MCP tool backed by an aggregate service, or is direct connection expected?
7. Data retention rules for the ledger and decision log.
8. Integration mechanism per source (API, MCP, events, or data replication), and who owns each connection.
9. Partner First: how do requests that originate there become visible to the platform?
10. Teams: confirm it is out of scope for v1 and which entry point (NexChat, CIS apps) covers those requests.
11. One orchestrator per BU on a shared schema, or one organization-wide agent? Determines tenancy and scoping.

Next Steps

- [ ] Confirm provisioning path and approvers with Dev Studio / DBA contacts (Q1 above).
- [ ] Walk Agent Bridge's DDL structure (NDP-5241) with its DP owner as a template.
- [ ] Inventory the sources each BU's Q2C requests touch, and pick an integration mechanism for each.
- [ ] Workshop with Q2C process owners across BUs to list curated entities and name owners.
- [ ] Draft v0 table definitions for `activity_ledger` and `decision_log` (the agent-written domains every workflow needs first: dedup, shared context, measurement).
- [ ] Decide curation UI approach.
- [ ] Update the diagram to show logic-controlled vs. business-curated writers.

Sources (Confluence)

Page	ID	Used for
BRD: Cisco Sales Email Auto-Triage v1.2 (APPP)	745641492	Requirements, OKRs, decisions D1–D8, open questions
00 How to use IDP	672273951	IDP scope, Dev Studio / idp-editor relationship

Page	ID	Used for
Auto Triage, Dev Studio Setup / Deployment	793518042 / 793518043	Component workflow, approval, NDP binding
003- Database Schema	605672539	Schema registration, Database Group
NDP-5241 Production Deployment Statement	782639356	DDL DO pattern, Shared vs. Disty schemas, Agent Bridge precedent
NDP-7665 / NDP-8129	771996257 / 785816890	Dev Studio MongoDB component
002 Event driven architecture - Database Decision	240338470	Decision-rule-managed tables precedent
MCP server for api-management	705296321	DB ownership / dependency discovery
Quick Start: new Dev Studio Component skill	771984648	Extending Dev Studio artifact types
Nexchat 101	578532945	NexChat as the approved CIS agent platform
Partner First as a Channel	693472139	Partner First as a partner-facing channel
NDP-7397 Process Mining Q2C POC	761919221	Existing Q2C visibility effort (Quote → Order → Ship → Invoice), possible overlap to check